

## PRÁCTICA 1

# Python, tiempos de ejecución, RLE, TAD Conjunto Disjunto y Componentes Conexas

Guía para entender qué hay que implementar, medir, justificar  
y entregar

Curso 2026-27



# La práctica en una mirada

Tres bloques conectados: implementar, experimentar y justificar.

## 1. Python y eficiencia

- `time_measure`
- `find_duplicates`
- `has_sum_pair`
- Medición de casos favorables y desfavorables

## 2. Compresión RLE

- `rle_encode_naive`
- `rle_encode_optimized`
- Comparación de tiempos
- Relación entre teoría y resultados

## 3. Conjuntos disjuntos y grafos

- `init_cd`, `union` y `find`
- `cd_2_dict`
- Componentes conexas con `ccs`
- Corrección y coste

# Reglas que afectan a la corrección automática

La interfaz forma parte de la solución.

## Nombres exactos

Respetar nombres de funciones, número y orden de parámetros y valores de retorno.

## Sin salida por pantalla

Elimina los print de depuración antes de entregar.

## Sin código ejecutable

El módulo debe poder importarse sin lanzar pruebas, lecturas o instrucciones adicionales.

## Cambios in place

Cuando se solicite, modifica la misma lista y devuelve la referencia requerida.

**Idea clave: una función correcta con una firma distinta puede no ser corregida por el script.**

## I.A.1 · time\_measure

Construir una función reutilizable para medir y resumir tiempos.

```
time_measure(f, dataprep, Nlist, Nrep=1000,  
Nstat=100)
```

### Entradas

f: función a medir  
dataprep: genera datos  
Nlist: tamaños n

### Repeticiones

Nrep: ejecuciones por  
medida  
Nstat: entradas distintas

### Salida

Una lista [(m0, v0), ..., (mk-1, vk-1)] con la media y la varianza para cada tamaño de Nlist.



# Cómo debe funcionar la medición

Separar preparación, repetición y estadística.



**Precaución: la preparación de datos no debe contaminar el tiempo de ejecución de f.**

## I.A.2 · find\_duplicates

Detectar duplicados sin repetirlos en el resultado.

`find_duplicates(lst) → list`

### Objetivo

Recibir una lista de enteros y devolver otra lista con los valores que aparecen más de una vez, sin duplicados en la salida y respetando el orden indicado por el enunciado.

**[2, 1, 1, 2, 1]**



**[1, 2]**

Cada elemento aparece una sola vez en el resultado.

## I.A.3 · has\_sum\_pair

Buscar dos posiciones distintas cuya suma sea target.

`has_sum_pair(lst, target) → bool`

### Condición

Existen índices distintos  $i$  y  $j$  tales que  $lst[i] + lst[j] == target$ .

### Eficiencia

Usar set o dict para alcanzar tiempo esperado  $O(n)$  y espacio adicional  $O(n)$ .

### Casos límite

Un único 5 no sirve para  $target=10$ ; dos valores 5 en posiciones distintas sí sirven.

**No se pide devolver la pareja: solo True o False.**

# Experimento con `has_sum_pair`

Medir cómo cambia el tiempo al crecer  $n$ .

## Caso favorable

Existe una pareja y se detecta pronto.

Pregunta: ¿el tiempo observado crece poco o depende de dónde aparece la pareja?

## Caso desfavorable

No existe una pareja válida.

La función debe examinar toda la lista antes de devolver False.

## Qué representar

Eje X: tamaño  $n$

Eje Y: tiempo medio

Incluir dispersión o varianza y explicar Nrep/Nstat.

**No basta con dibujar la gráfica: hay que justificar el comportamiento observado.**

## I.B · Run-Length Encoding (RLE)

Comprimir rachas consecutivas, no apariciones globales.

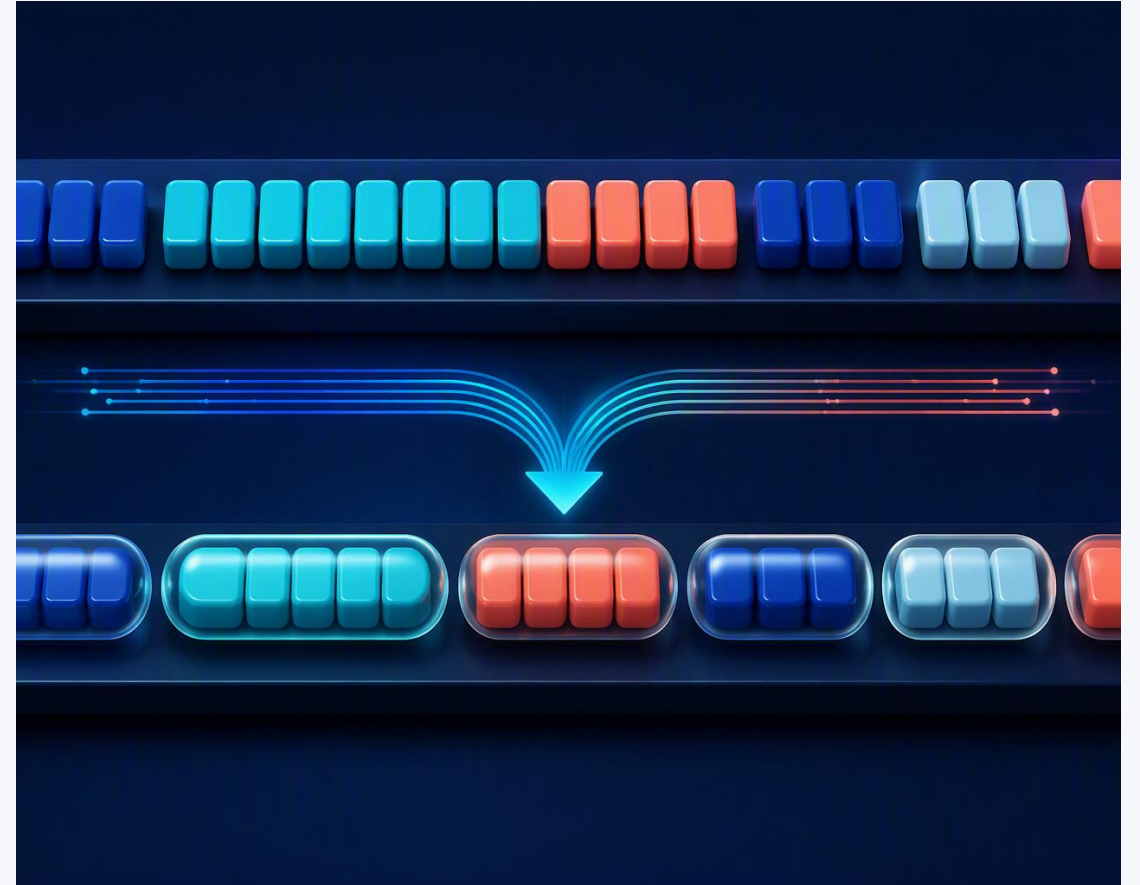
Entrada

`['A', 'A', 'A', 'B', 'B', 'C', 'A']`



`[('A', 3), ('B', 2), ('C', 1), ('A', 1)]`

La última A forma una racha nueva.



# Dos implementaciones de RLE

Mismo resultado funcional, distinto coste de construcción.

## `rle_encode_naive`

- Recorrido secuencial
- Construcción mediante concatenaciones sucesivas con +
- Las copias repetidas pueden encarecer el algoritmo
- Analizar el peor caso

## `rle_encode_optimized`

- Un único recorrido
- Construcción mediante append
- Tiempo esperado  $O(n)$
- Salida equivalente a la versión naive

**Comparar entradas con pocas rachas y con muchas rachas.**

## II.A · TAD Conjunto Disjunto

Mantener una partición de  $\{0, 1, \dots, n-1\}$  en subconjuntos disjuntos.

### Representación

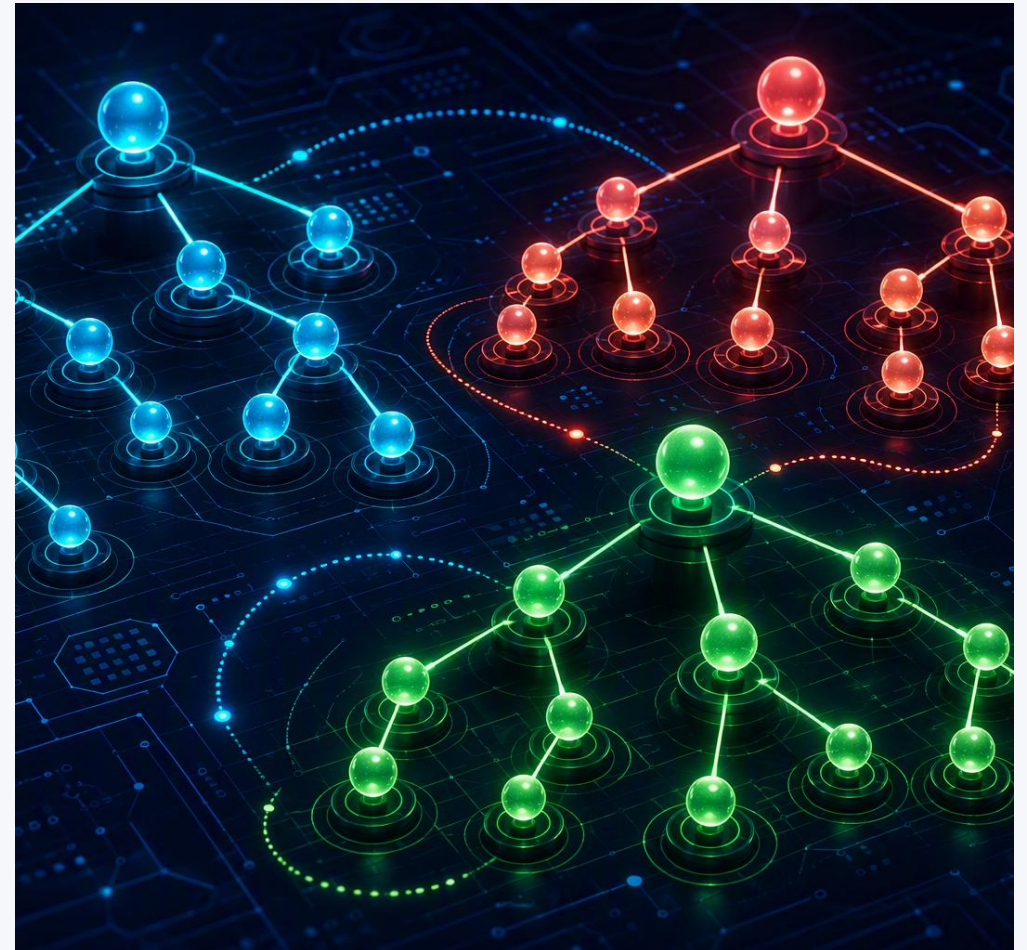
Array NumPy con padres o rangos negativos.

### Unión por rangos

Al unir dos representantes, el árbol de menor rango se enlaza al de mayor rango.

### Compresión de caminos

find actualiza el camino para acercar los nodos al representante.



# Funciones del TAD Conjunto Disjunto

Firmas exactas y responsabilidad de cada operación.

**`init_cd(n: int) → np.ndarray`**

Crea un array de longitud `n` con valor `-1` en todas sus posiciones.

**`union(rep_1, rep_2, p_cd) → int`**

Une por rangos y devuelve el representante del conjunto resultante.

**`find(ind, p_cd) → int`**

Devuelve el representante de `ind` y realiza compresión de caminos.

**`cd_2_dict(p_cd) → Dict`**

Convierte el CD en un diccionario: representante → lista de miembros.

## Invariante esencial

Cada índice pertenece a un único subconjunto y cada subconjunto tiene un representante.

## II.B · Componentes conexas

Aplicar el TAD Conjunto Disjunto a un grafo no dirigido.

### Entrada del grafo

$n$  vértices:  $[0, \dots, n-1]$

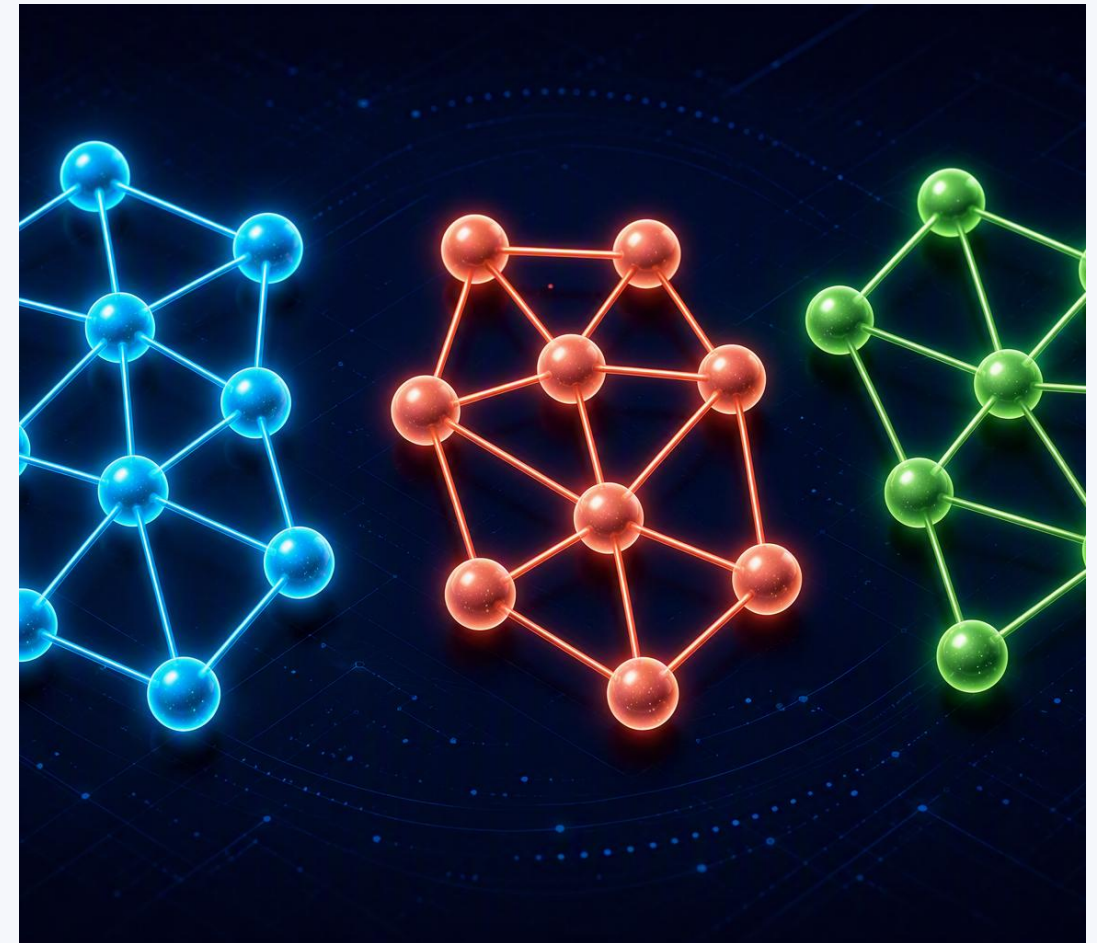
Lista  $l$  de ramas  $(u, v)$

### Idea del algoritmo

Inicializar el CD y procesar cada rama. Si sus extremos están en subconjuntos distintos, unirlos.

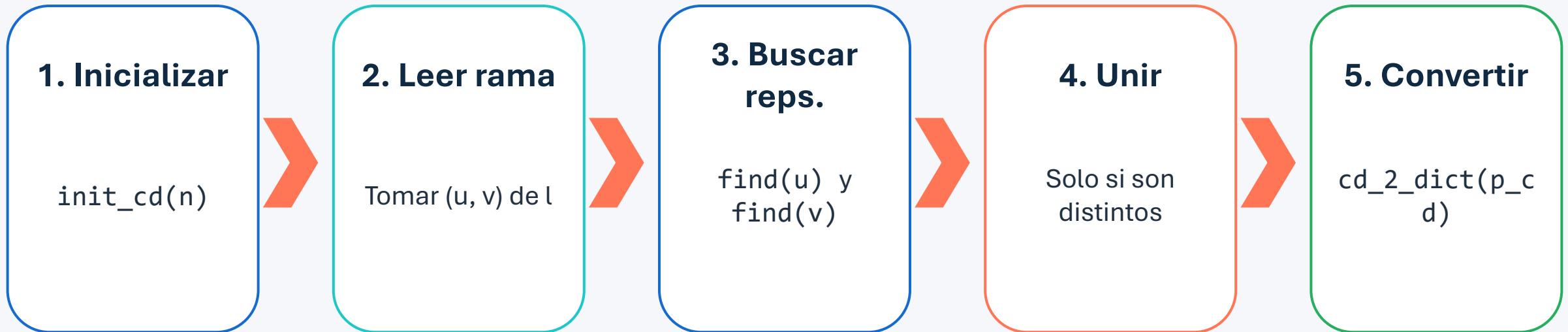
### Resultado

Convertir la tabla del CD a un dict con las componentes conexas.



## II.B.1 · $\text{ccs}(n, l) \rightarrow \text{Dict}$

Flujo de procesamiento de las ramas del grafo.



**Las ramas repetidas o invertidas no deben alterar la partición final.**

## II.C · Cuestiones razonadas

La memoria debe justificar resultado, corrección y coste.

### 1. Ramas repetidas o invertidas

Explica si afectan al resultado y por qué la operación de unión conserva la partición.

### 2. Corrección del algoritmo

Argumenta la correspondencia entre subconjuntos disjuntos y componentes conexas.

### 3. Coste en función de $n$ y $|I|$

Separa el coste de inicialización, procesamiento de ramas y conversión final.

### Qué se espera en la respuesta

No basta con indicar una complejidad: hay que justificar las operaciones consideradas.

**Conecta la estructura del algoritmo con la propiedad de conectividad del grafo.**

# Qué hay que entregar

Dos ficheros, con formatos y contenidos distintos.

## p1.py

- `time_measure`
- `find_duplicates`
- `has_sum_pair`
- `rle_encode_naive`
- `rle_encode_optimized`
- `init_cd, union, find`
- `cd_2_dict` y `ccs`

## measure.pdf

- Gráficas de `has_sum_pair`
- Comparativa RLE
- Metodología de medición
- Valores de `Nlist`, `Nrep` y `Nstat`
- Ejes y unidades
- Análisis razonado de resultados

# Checklist antes de entregar

- ✓ Las firmas coinciden exactamente con el enunciado: las funciones devuelven el tipo y la estructura solicitados.
- ✓ No hay print ni código de prueba ejecutándose al importar.
- ✓ Los cambios in place conservan la lista correcta.
- ✓ `union` usa rangos y `find` aplica compresión de caminos.
- ✓ `ccs` considera todos los vértices, incluidos los aislados.
- ✓ Las gráficas tienen ejes, unidades y explicación.
- ✓ El análisis conecta resultados experimentales y complejidad teórica.
- ✓ Se penalizarán las faltas de ortografía/erratas encontradas en la memoria (-0,5p por cada 3 faltas/erratas).

**Objetivo: código correcto, memoria argumentada y entrega completada.**